

fix-hostfiles

1.20

Generated by Doxygen 1.18.0

1 Data Structure Index	1
1.1 Data Structures	1
2 File Index	3
2.1 File List	3
3 Data Structure Documentation	5
3.1 Command Struct Reference	5
3.1.1 Detailed Description	5
3.1.2 Field Documentation	5
3.1.2.1 action	5
3.1.2.2 dns_name	5
3.1.2.3 verbose	6
3.2 Paths Struct Reference	6
3.2.1 Detailed Description	6
3.2.2 Field Documentation	6
3.2.2.1 allow_file	6
3.2.2.2 etc_dir	6
3.2.2.3 hblock_dir	7
3.2.2.4 hosts_file	7
3.2.2.5 original_hosts_file	7
4 File Documentation	9
4.1 src/actions.h File Reference	9
4.1.1 Detailed Description	9
4.1.2 Enumeration Type Documentation	10
4.1.2.1 Action	10
4.2 actions.h	10
4.3 src/fix-hosts.c File Reference	10
4.3.1 Detailed Description	11
4.3.2 Function Documentation	11
4.3.2.1 addDnsName()	11
4.3.2.2 flushDnsCache()	12
4.3.2.3 joinPath()	12
4.3.2.4 lineContainsHost()	13
4.3.2.5 loadPaths()	13
4.3.2.6 removeHostEntry()	14
4.3.2.7 showFileInfo()	14
4.3.2.8 updateHostsFiles()	15
4.3.2.9 usage()	15

4.4 src/fix-hosts.h File Reference	15
4.4.1 Detailed Description	16
4.4.2 Macro Definition Documentation	16
4.4.2.1 PATH_MAX	16
4.4.3 Function Documentation	16
4.4.3.1 addDnsName()	16
4.4.3.2 flushDnsCache()	17
4.4.3.3 loadPaths()	17
4.4.3.4 removeHostEntry()	18
4.4.3.5 updateHostsFiles()	18
4.4.3.6 usage()	19
4.5 fix-hosts.h	19
4.6 src/main.c File Reference	19
4.6.1 Detailed Description	20
4.6.2 Function Documentation	20
4.6.2.1 main()	20
4.6.2.2 parseArguments()	21
4.6.2.3 programName()	21
4.6.2.4 selectAction()	21
4.7 src/system-actions.c File Reference	22
4.7.1 Detailed Description	23
4.7.2 Function Documentation	23
4.7.2.1 appendLine()	23
4.7.2.2 confirm()	23
4.7.2.3 copyFile()	24
4.7.2.4 ensureDirectory()	24
4.7.2.5 fileContainsLine()	24
4.7.2.6 fileExists()	25
4.7.2.7 fileInfo()	25
4.7.2.8 fileType()	26
4.7.2.9 formatPermissions()	26
4.7.2.10 isValidDnsName()	26
4.7.2.11 listFiles()	27
4.7.2.12 reportError()	27
4.7.2.13 runCommand()	27
4.8 src/system-actions.h File Reference	28
4.8.1 Detailed Description	28
4.8.2 Macro Definition Documentation	28
4.8.2.1 REPORT_ERROR	28

4.8.3 Function Documentation	29
4.8.3.1 appendLine()	29
4.8.3.2 confirm()	29
4.8.3.3 copyFile()	29
4.8.3.4 ensureDirectory()	30
4.8.3.5 fileContainsLine()	30
4.8.3.6 fileExists()	31
4.8.3.7 fileInfo()	31
4.8.3.8 isValidDnsName()	31
4.8.3.9 listFiles()	32
4.8.3.10 reportError()	32
4.8.3.11 runCommand()	33
4.9 system-actions.h	33
Index	35

Chapter 1

Data Structure Index

1.1 Data Structures

Here are the data structures with brief descriptions:

Command	5
Paths	6

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

src/ actions.h	
Command actions and parsed command-line state	9
src/ fix-hosts.c	
Application-level hosts-file operations	10
src/ fix-hosts.h	
Hosts-file paths and application operations	15
src/ main.c	
Command-line parsing and application dispatch	19
src/ system-actions.c	
Filesystem, subprocess, validation, and reporting utilities	22
src/ system-actions.h	
Reusable filesystem, process, validation, and reporting helpers	28

Chapter 3

Data Structure Documentation

3.1 Command Struct Reference

```
#include <actions.h>
```

Data Fields

- [Action](#) `action`
- `const char *` [dns_name](#)
- `bool` [verbose](#)

3.1.1 Detailed Description

Parsed command-line request.

3.1.2 Field Documentation

3.1.2.1 `action`

[Action](#) `action`

Operation to perform.

3.1.2.2 `dns_name`

`const char* dns_name`

Domain supplied to ACTION_ADD; otherwise NULL.

3.1.2.3 verbose

```
bool verbose
```

Whether to display detailed paths, commands, and file metadata.

The documentation for this struct was generated from the following file:

- [src/actions.h](#)

3.2 Paths Struct Reference

```
#include <fix-hosts.h>
```

Data Fields

- char [etc_dir](#) [PATH_MAX]
- char [hosts_file](#) [PATH_MAX]
- char [original_hosts_file](#) [PATH_MAX]
- char [hblock_dir](#) [PATH_MAX]
- char [allow_file](#) [PATH_MAX]

3.2.1 Detailed Description

Filesystem paths used by hosts-file operations.

3.2.2 Field Documentation

3.2.2.1 allow_file

```
char allow_file[PATH_MAX]
```

hblock DNS allow list.

3.2.2.2 etc_dir

```
char etc_dir[PATH_MAX]
```

Directory containing the system hosts files.

3.2.2.3 hblock_dir

```
char hblock_dir[PATH_MAX]
```

Directory containing hblock configuration.

3.2.2.4 hosts_file

```
char hosts_file[PATH_MAX]
```

Active hosts file.

3.2.2.5 original_hosts_file

```
char original_hosts_file[PATH_MAX]
```

Backup used by prep and restore.

The documentation for this struct was generated from the following file:

- [src/fix-hosts.h](#)

Chapter 4

File Documentation

4.1 src/actions.h File Reference

[Command](#) actions and parsed command-line state.

```
#include <stdbool.h>
```

Data Structures

- struct [Command](#)

Enumerations

- enum [Action](#) {
 [ACTION_INVALID](#) = 0 , [ACTION_PREP](#) , [ACTION_RESTORE](#) , [ACTION_ADD](#) ,
 [ACTION_FLUSH](#) }

4.1.1 Detailed Description

[Command](#) actions and parsed command-line state.

4.1.2 Enumeration Type Documentation

4.1.2.1 Action

enum `Action`

Actions supported by the command-line interface.

Enumerator

<code>ACTION_INVALID</code>	No action was selected, or parsing failed.
<code>ACTION_PREP</code>	Back up the active hosts file and run hblock.
<code>ACTION_RESTORE</code>	Restore the active hosts file from its original backup.
<code>ACTION_ADD</code>	Add a DNS name to the hblock allow list and unblock it.
<code>ACTION_FLUSH</code>	Flush the macOS DNS cache and restart mDNSResponder.

4.2 actions.h

[Go to the documentation of this file.](#)

```

00001
00005
00006 #ifndef ACTIONS_H
00007 #define ACTIONS_H
00008
00009 #include <stdbool.h>
00010
00012 typedef enum {
00014     ACTION_INVALID = 0,
00016     ACTION_PREP,
00018     ACTION_RESTORE,
00020     ACTION_ADD,
00022     ACTION_FLUSH
00023 } Action;
00024
00026 typedef struct {
00028     Action action;
00030     const char *dns_name;
00032     bool verbose;
00033 } Command;
00034
00035 #endif

```

4.3 src/fix-hosts.c File Reference

Application-level hosts-file operations.

```

#include "fix-hosts.h"
#include "system-actions.h"
#include <ctype.h>
#include <errno.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <sys/utsname.h>
#include <unistd.h>

```

Functions

- static int [joinPath](#) (char *destination, size_t capacity, const char *directory, const char *name)
- void [usage](#) (const char *program, FILE *stream)
- int [loadPaths](#) ([Paths](#) *paths)
- static int [showFileInfo](#) (const char *label, const char *path)
- int [updateHostsFiles](#) (const [Paths](#) *paths, [Action](#) action, bool verbose)
- static bool [lineContainsHost](#) (const char *line, const char *dns_name)
- int [removeHostEntry](#) (const char *hosts_path, const char *dns_name)
- int [addDnsName](#) (const [Paths](#) *paths, const char *dns_name, bool verbose)
- int [flushDnsCache](#) (bool verbose)

4.3.1 Detailed Description

Application-level hosts-file operations.

4.3.2 Function Documentation

4.3.2.1 [addDnsName\(\)](#)

```
int addDnsName (  
    const Paths * paths,  
    const char * dns_name,  
    bool verbose)
```

Add a DNS name to hblock's allow list and unblock it.

The allow-list update is idempotent. Afterward, matching hosts-file lines are removed through [removeHostEntry\(\)](#).

Parameters

<i>paths</i>	Filesystem paths used by the operation.
<i>dns_name</i>	DNS name to add and unblock.
<i>verbose</i>	Whether to display detailed before-and-after file metadata.

Returns

EXIT_SUCCESS on success; EXIT_FAILURE on validation or I/O error.

4.3.2.2 flushDnsCache()

```
int flushDnsCache (  
    bool verbose)
```

Flush the macOS DNS cache and restart mDNSResponder.

Commands are executed directly without invoking a shell. The function also verifies that mDNSResponder is present after the restart.

Parameters

<i>verbose</i>	Whether to display platform and command details.
----------------	--

Returns

EXIT_SUCCESS when all commands succeed and the daemon is found; EXIT_FAILURE on unsupported platforms or command failure.

4.3.2.3 joinPath()

```
int joinPath (  
    char * destination,  
    size_t capacity,  
    const char * directory,  
    const char * name) [static]
```

Join one directory and leaf name with exactly one slash.

Parameters

<i>destination</i>	Buffer that receives the joined path.
<i>capacity</i>	Size of destination in bytes.
<i>directory</i>	Directory portion of the path.
<i>name</i>	Final pathname component.

Returns

EXIT_SUCCESS on success; EXIT_FAILURE when the path does not fit.

4.3.2.4 lineContainsHost()

```
bool lineContainsHost (  
    const char * line,  
    const char * dns_name) [static]
```

Determine whether a hosts-file line contains an exact DNS token.

Tokens are separated by whitespace; comments beginning with # are ignored.

Parameters

<i>line</i>	Complete hosts-file line.
<i>dns_name</i>	DNS token to locate.

Returns

true when an exact token is present before any comment.

4.3.2.5 loadPaths()

```
int loadPaths (  
    Paths * paths)
```

Load all filesystem paths used by the application.

Production paths are derived from /etc. The FIX_HOSTFILES_ETC_DIR and FIX_HOSTFILES_HBLOCK_DIR environment variables provide explicit overrides for isolated tests and development fixtures.

Parameters

<i>paths</i>	Structure populated with validated, absolute or relative paths.
--------------	---

Returns

EXIT_SUCCESS on success; EXIT_FAILURE if a path exceeds PATH_MAX.

4.3.2.6 removeHostEntry()

```
int removeHostEntry (  
    const char * hosts_path,  
    const char * dns_name)
```

Atomically remove hosts-file lines containing an exact DNS token.

The original file is first copied to a .bak file. Retained lines are written to a same-directory temporary file, synchronized, assigned the original permissions and ownership, and atomically renamed over the hosts file.

Parameters

<i>hosts_path</i>	Path to the active hosts file.
<i>dns_name</i>	Validated DNS token to remove.

Returns

EXIT_SUCCESS on success; EXIT_FAILURE on any filesystem error.

4.3.2.7 showFileInfo()

```
int showFileInfo (  
    const char * label,  
    const char * path) [static]
```

Display metadata for a path when it exists.

Parameters

<i>label</i>	Heading that identifies the path's role.
<i>path</i>	Filesystem path to inspect.

Returns

EXIT_SUCCESS when absent or successfully inspected; otherwise EXIT_FAILURE.

4.3.2.8 updateHostsFiles()

```
int updateHostsFiles (
    const Paths * paths,
    Action action,
    bool verbose)
```

Back up or restore the system hosts file.

Existing destination files require interactive confirmation. ACTION_PREP runs hblock after creating the backup; ACTION_RESTORE copies the backup over the active hosts file.

Parameters

<i>paths</i>	Filesystem paths used by the operation.
<i>action</i>	ACTION_PREP or ACTION_RESTORE.
<i>verbose</i>	Whether to display detailed before-and-after file metadata.

Returns

EXIT_SUCCESS on success or user cancellation; EXIT_FAILURE on error.

4.3.2.9 usage()

```
void usage (
    const char * program,
    FILE * stream)
```

Print command help.

Parameters

<i>program</i>	Basename used to invoke the executable.
<i>stream</i>	Destination stream, normally stdout or stderr.

4.4 src/fix-hosts.h File Reference

Hosts-file paths and application operations.

```
#include "actions.h"
#include <limits.h>
#include <stdio.h>
```

Data Structures

- struct [Paths](#)

Macros

- `#define` [PATH_MAX](#) 4096

Functions

- void [usage](#) (const char *program, FILE *stream)
- int [loadPaths](#) ([Paths](#) *paths)
- int [updateHostsFiles](#) (const [Paths](#) *paths, [Action](#) action, bool verbose)
- int [addDnsName](#) (const [Paths](#) *paths, const char *dns_name, bool verbose)
- int [removeHostEntry](#) (const char *hosts_path, const char *dns_name)
- int [flushDnsCache](#) (bool verbose)

4.4.1 Detailed Description

Hosts-file paths and application operations.

4.4.2 Macro Definition Documentation

4.4.2.1 PATH_MAX

```
#define PATH_MAX 4096
```

Fallback maximum path length for platforms that do not define `PATH_MAX`.

4.4.3 Function Documentation

4.4.3.1 addDnsName()

```
int addDnsName (  
    const Paths * paths,  
    const char * dns_name,  
    bool verbose)
```

Add a DNS name to hblock's allow list and unblock it.

The allow-list update is idempotent. Afterward, matching hosts-file lines are removed through [removeHostEntry\(\)](#).

Parameters

<i>paths</i>	Filesystem paths used by the operation.
<i>dns_name</i>	DNS name to add and unblock.
<i>verbose</i>	Whether to display detailed before-and-after file metadata.

Returns

EXIT_SUCCESS on success; EXIT_FAILURE on validation or I/O error.

4.4.3.2 flushDnsCache()

```
int flushDnsCache (  
    bool verbose)
```

Flush the macOS DNS cache and restart mDNSResponder.

Commands are executed directly without invoking a shell. The function also verifies that mDNSResponder is present after the restart.

Parameters

<i>verbose</i>	Whether to display platform and command details.
----------------	--

Returns

EXIT_SUCCESS when all commands succeed and the daemon is found; EXIT_FAILURE on unsupported platforms or command failure.

4.4.3.3 loadPaths()

```
int loadPaths (  
    Paths * paths)
```

Load all filesystem paths used by the application.

Production paths are derived from /etc. The FIX_HOSTFILES_ETC_DIR and FIX_HOSTFILES_HBLOCK_DIR environment variables provide explicit overrides for isolated tests and development fixtures.

Parameters

<i>paths</i>	Structure populated with validated, absolute or relative paths.
--------------	---

Returns

EXIT_SUCCESS on success; EXIT_FAILURE if a path exceeds PATH_MAX.

4.4.3.4 removeHostEntry()

```
int removeHostEntry (
    const char * hosts_path,
    const char * dns_name)
```

Atomically remove hosts-file lines containing an exact DNS token.

The original file is first copied to a .bak file. Retained lines are written to a same-directory temporary file, synchronized, assigned the original permissions and ownership, and atomically renamed over the hosts file.

Parameters

<i>hosts_path</i>	Path to the active hosts file.
<i>dns_name</i>	Validated DNS token to remove.

Returns

EXIT_SUCCESS on success; EXIT_FAILURE on any filesystem error.

4.4.3.5 updateHostsFiles()

```
int updateHostsFiles (
    const Paths * paths,
    Action action,
    bool verbose)
```

Back up or restore the system hosts file.

Existing destination files require interactive confirmation. ACTION_PREP runs hblock after creating the backup; ACTION_RESTORE copies the backup over the active hosts file.

Parameters

<i>paths</i>	Filesystem paths used by the operation.
<i>action</i>	ACTION_PREP or ACTION_RESTORE.
<i>verbose</i>	Whether to display detailed before-and-after file metadata.

Returns

EXIT_SUCCESS on success or user cancellation; EXIT_FAILURE on error.

4.4.3.6 usage()

```
void usage (
    const char * program,
    FILE * stream)
```

Print command help.

Parameters

<i>program</i>	Basename used to invoke the executable.
<i>stream</i>	Destination stream, normally stdout or stderr.

4.5 fix-hosts.h

[Go to the documentation of this file.](#)

```
00001
00005
00006 #ifndef FIX_HOSTS_H
00007 #define FIX_HOSTS_H
00008
00009 #include "actions.h"
00010
00011 #include <limits.h>
00012 #include <stdio.h>
00013
00014 #ifndef PATH_MAX
00016 #define PATH_MAX 4096
00017 #endif
00018
00020 typedef struct {
00022     char etc_dir[PATH_MAX];
00024     char hosts_file[PATH_MAX];
00026     char original_hosts_file[PATH_MAX];
00028     char hblock_dir[PATH_MAX];
00030     char allow_file[PATH_MAX];
00031 } Paths;
00032
00039 void usage(const char *program, FILE *stream);
00040
00051 int loadPaths(Paths *paths);
00052
00065 int updateHostsFiles(const Paths *paths, Action action, bool verbose);
00066
00078 int addDnsName(const Paths *paths, const char *dns_name, bool verbose);
00079
00091 int removeHostEntry(const char *hosts_path, const char *dns_name);
00092
00103 int flushDnsCache(bool verbose);
00104
00105 #endif
```

4.6 src/main.c File Reference

Command-line parsing and application dispatch.

```
#include "actions.h"
#include "fix-hosts.h"
#include "system-actions.h"
#include <getopt.h>
```

```
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- static const char * [programName](#) (const char *path)
- static int [selectAction](#) ([Command](#) *command, [Action](#) action)
- static int [parseArguments](#) (int argc, char **argv, [Command](#) *command)
- int [main](#) (int argc, char **argv)

4.6.1 Detailed Description

Command-line parsing and application dispatch.

4.6.2 Function Documentation

4.6.2.1 [main\(\)](#)

```
int main (
    int argc,
    char ** argv)
```

Parse the requested operation, load its paths, and dispatch it.

Parameters

<i>argc</i>	Number of command-line arguments.
<i>argv</i>	NULL-terminated command-line argument vector.

Returns

EXIT_SUCCESS on success; EXIT_FAILURE on parsing or operation errors.

4.6.2.2 parseArguments()

```
int parseArguments (  
    int argc,  
    char ** argv,  
    Command * command) [static]
```

Parse short options, long options, and positional actions.

Parameters

<i>argc</i>	Argument count from main() .
<i>argv</i>	Argument vector from main() .
<i>command</i>	Output command populated from the arguments.

Returns

0 when execution should continue, 1 after displaying help, or -1 when the command line is invalid.

4.6.2.3 programName()

```
const char * programName (  
    const char * path) [static]
```

Return the final pathname component used to invoke the program.

Parameters

<i>path</i>	argv[0] from main() .
-------------	---------------------------------------

Returns

Pointer into path at its basename.

4.6.2.4 selectAction()

```
int selectAction (  
    Command * command,  
    Action action) [static]
```

Select one action while rejecting conflicting command syntax.

Parameters

<i>command</i>	Parsed command being populated.
<i>action</i>	Newly requested action.

Returns

EXIT_SUCCESS when selected; EXIT_FAILURE if an action already exists.

4.7 src/system-actions.c File Reference

Filesystem, subprocess, validation, and reporting utilities.

```
#include "system-actions.h"
#include <dirent.h>
#include <errno.h>
#include <fnmatch.h>
#include <grp.h>
#include <pwd.h>
#include <regex.h>
#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>
```

Functions

- void [reportError](#) (const char *file, const char *function, int line, const char *format,...)
- bool [confirm](#) (const char *prompt)
- bool [fileExists](#) (const char *path)
- int [copyFile](#) (const char *source_path, const char *destination_path)
- int [listFiles](#) (const char *directory_path, const char *pattern)
- static const char * [fileType](#) (mode_t mode)
- static void [formatPermissions](#) (mode_t mode, char permissions[11])
- int [fileInfo](#) (const char *path)
- int [ensureDirectory](#) (const char *path)
- int [runCommand](#) (char *const arguments[])
- bool [isValidDnsName](#) (const char *dns_name)
- bool [fileContainsLine](#) (const char *path, const char *expected_line)
- int [appendLine](#) (const char *path, const char *line)

4.7.1 Detailed Description

Filesystem, subprocess, validation, and reporting utilities.

4.7.2 Function Documentation

4.7.2.1 appendLine()

```
int appendLine (  
    const char * path,  
    const char * line)
```

Append one newline-terminated line to a file.

Parameters

<i>path</i>	File to create or append to.
<i>line</i>	Text to append without a terminating newline.

Returns

EXIT_SUCCESS on success; EXIT_FAILURE on an I/O error.

4.7.2.2 confirm()

```
bool confirm (  
    const char * prompt)
```

Repeatedly prompt for a yes-or-no response.

Parameters

<i>prompt</i>	Question displayed before the input prompt.
---------------	---

Returns

true for yes; false for no or end-of-file.

4.7.2.3 copyFile()

```
int copyFile (  
    const char * source_path,  
    const char * destination_path)
```

Copy a regular file and preserve its permission bits.

Parameters

<i>source_path</i>	Existing file to copy.
<i>destination_path</i>	Destination to create or overwrite.

Returns

EXIT_SUCCESS on success; EXIT_FAILURE on any I/O error.

4.7.2.4 ensureDirectory()

```
int ensureDirectory (  
    const char * path)
```

Create a directory if it does not already exist.

Parameters

<i>path</i>	Directory to validate or create.
-------------	----------------------------------

Returns

EXIT_SUCCESS when a directory exists at path; otherwise EXIT_FAILURE.

4.7.2.5 fileContainsLine()

```
bool fileContainsLine (  
    const char * path,  
    const char * expected_line)
```

Search a text file for an exact line.

Line endings are removed before comparison.

Parameters

<i>path</i>	File to search.
<i>expected_line</i>	Complete line to locate, without a newline.

Returns

true when the line is found; false otherwise.

4.7.2.6 fileExists()

```
bool fileExists (  
    const char * path)
```

Test whether a filesystem path exists.

Parameters

<i>path</i>	Path to inspect.
-------------	------------------

Returns

true when access(path, F_OK) succeeds.

4.7.2.7 fileInfo()

```
int fileInfo (  
    const char * path)
```

Display detailed metadata for one filesystem object.

The report includes type, permissions, ownership, size, inode, link count, and last-modification time. Symbolic links are inspected without following their targets.

Parameters

<i>path</i>	Filesystem object to inspect.
-------------	-------------------------------

Returns

EXIT_SUCCESS on success; EXIT_FAILURE when metadata is unavailable.

4.7.2.8 fileType()

```
const char * fileType (
    mode_t mode) [static]
```

Return a readable name for a mode's filesystem-object type.

Parameters

<i>mode</i>	st_mode value returned by lstat(2).
-------------	-------------------------------------

Returns

Static string describing the object type.

4.7.2.9 formatPermissions()

```
void formatPermissions (
    mode_t mode,
    char permissions[11]) [static]
```

Format a mode as the familiar ten-character ls-style permission string.

Parameters

<i>mode</i>	st_mode value returned by lstat(2).
<i>permissions</i>	Output buffer with room for eleven bytes.

4.7.2.10 isValidDnsName()

```
bool isValidDnsName (
    const char * dns_name)
```

Validate a fully qualified DNS name.

Labels may contain letters, numbers, and interior hyphens. The final label must contain at least two letters.

Parameters

<i>dns_name</i>	Candidate DNS name.
-----------------	---------------------

Returns

true when the complete string matches the accepted DNS syntax.

4.7.2.11 listFiles()

```
int listFiles (
    const char * directory_path,
    const char * pattern)
```

List directory entries matching a shell-style pattern.

Parameters

<i>directory_path</i>	Directory to scan.
<i>pattern</i>	Pattern interpreted by fnmatch(3).

Returns

EXIT_SUCCESS on success; EXIT_FAILURE on directory or stat errors.

4.7.2.12 reportError()

```
void reportError (
    const char * file,
    const char * function,
    int line,
    const char * format,
    ...)
```

Report a formatted error with its source location.

Parameters

<i>file</i>	Translation unit supplied by REPORT_ERROR.
<i>function</i>	Calling function supplied by REPORT_ERROR.
<i>line</i>	Source line supplied by REPORT_ERROR.
<i>format</i>	printf-compatible message format.

4.7.2.13 runCommand()

```
int runCommand (
    char *const arguments[])
```

Execute an argument vector without invoking a shell.

The function forks, calls execvp(3) in the child, and waits for that child.

Parameters

<i>arguments</i>	NULL-terminated argv array whose first element is a command.
------------------	--

Returns

The child's exit status, or EXIT_FAILURE when it cannot be executed.

4.8 src/system-actions.h File Reference

Reusable filesystem, process, validation, and reporting helpers.

```
#include <stdbool.h>
```

Macros

- #define [REPORT_ERROR\(...\)](#)

Functions

- void [reportError](#) (const char *file, const char *function, int line, const char *format,...)
- bool [confirm](#) (const char *prompt)
- bool [fileExists](#) (const char *path)
- int [copyFile](#) (const char *source_path, const char *destination_path)
- int [listFiles](#) (const char *directory_path, const char *pattern)
- int [fileInfo](#) (const char *path)
- int [ensureDirectory](#) (const char *path)
- int [runCommand](#) (char *const arguments[])
- bool [isValidDnsName](#) (const char *dns_name)
- bool [fileContainsLine](#) (const char *path, const char *expected_line)
- int [appendLine](#) (const char *path, const char *line)

4.8.1 Detailed Description

Reusable filesystem, process, validation, and reporting helpers.

4.8.2 Macro Definition Documentation

4.8.2.1 REPORT_ERROR

```
#define REPORT_ERROR(  
    ...)
```

Value:

```
reportError(__FILE__, __func__, __LINE__, __VA_ARGS__)
```

Report an error with the calling file, function, and line number.

4.8.3 Function Documentation

4.8.3.1 appendLine()

```
int appendLine (  
    const char * path,  
    const char * line)
```

Append one newline-terminated line to a file.

Parameters

<i>path</i>	File to create or append to.
<i>line</i>	Text to append without a terminating newline.

Returns

EXIT_SUCCESS on success; EXIT_FAILURE on an I/O error.

4.8.3.2 confirm()

```
bool confirm (  
    const char * prompt)
```

Repeatedly prompt for a yes-or-no response.

Parameters

<i>prompt</i>	Question displayed before the input prompt.
---------------	---

Returns

true for yes; false for no or end-of-file.

4.8.3.3 copyFile()

```
int copyFile (  
    const char * source_path,  
    const char * destination_path)
```

Copy a regular file and preserve its permission bits.

Parameters

<i>source_path</i>	Existing file to copy.
<i>destination_path</i>	Destination to create or overwrite.

Returns

EXIT_SUCCESS on success; EXIT_FAILURE on any I/O error.

4.8.3.4 ensureDirectory()

```
int ensureDirectory (  
    const char * path)
```

Create a directory if it does not already exist.

Parameters

<i>path</i>	Directory to validate or create.
-------------	----------------------------------

Returns

EXIT_SUCCESS when a directory exists at path; otherwise EXIT_FAILURE.

4.8.3.5 fileContainsLine()

```
bool fileContainsLine (  
    const char * path,  
    const char * expected_line)
```

Search a text file for an exact line.

Line endings are removed before comparison.

Parameters

<i>path</i>	File to search.
<i>expected_line</i>	Complete line to locate, without a newline.

Returns

true when the line is found; false otherwise.

4.8.3.6 fileExists()

```
bool fileExists (  
    const char * path)
```

Test whether a filesystem path exists.

Parameters

<i>path</i>	Path to inspect.
-------------	------------------

Returns

true when access(path, F_OK) succeeds.

4.8.3.7 fileInfo()

```
int fileInfo (  
    const char * path)
```

Display detailed metadata for one filesystem object.

The report includes type, permissions, ownership, size, inode, link count, and last-modification time. Symbolic links are inspected without following their targets.

Parameters

<i>path</i>	Filesystem object to inspect.
-------------	-------------------------------

Returns

EXIT_SUCCESS on success; EXIT_FAILURE when metadata is unavailable.

4.8.3.8 isValidDnsName()

```
bool isValidDnsName (  
    const char * dns_name)
```

Validate a fully qualified DNS name.

Labels may contain letters, numbers, and interior hyphens. The final label must contain at least two letters.

Parameters

<i>dns_name</i>	Candidate DNS name.
-----------------	---------------------

Returns

true when the complete string matches the accepted DNS syntax.

4.8.3.9 listFiles()

```
int listFiles (
    const char * directory_path,
    const char * pattern)
```

List directory entries matching a shell-style pattern.

Parameters

<i>directory_path</i>	Directory to scan.
<i>pattern</i>	Pattern interpreted by fnmatch(3).

Returns

EXIT_SUCCESS on success; EXIT_FAILURE on directory or stat errors.

4.8.3.10 reportError()

```
void reportError (
    const char * file,
    const char * function,
    int line,
    const char * format,
    ...)
```

Report a formatted error with its source location.

Parameters

<i>file</i>	Translation unit supplied by REPORT_ERROR.
<i>function</i>	Calling function supplied by REPORT_ERROR.
<i>line</i>	Source line supplied by REPORT_ERROR.
<i>format</i>	printf-compatible message format.

4.8.3.11 runCommand()

```
int runCommand (
    char *const arguments[])
```

Execute an argument vector without invoking a shell.

The function forks, calls `execvp(3)` in the child, and waits for that child.

Parameters

<i>arguments</i>	NULL-terminated argv array whose first element is a command.
------------------	--

Returns

The child's exit status, or `EXIT_FAILURE` when it cannot be executed.

4.9 system-actions.h

[Go to the documentation of this file.](#)

```
00001
00005
00006 #ifndef SYSTEM_ACTIONS_H
00007 #define SYSTEM_ACTIONS_H
00008
00009 #include <stdbool.h>
00010
00012 #define REPORT_ERROR(...) reportError(__FILE__, __func__, __LINE__, __VA_ARGS__)
00013
00022 void reportError(const char *file, const char *function, int line,
00023                 const char *format, ...);
00024
00031 bool confirm(const char *prompt);
00032
00039 bool fileExists(const char *path);
00040
00048 int copyFile(const char *source_path, const char *destination_path);
00049
00057 int listFiles(const char *directory_path, const char *pattern);
00058
00069 int fileInfo(const char *path);
00070
00077 int ensureDirectory(const char *path);
00078
00087 int runCommand(char *const arguments[]);
00088
00098 bool isValidDnsName(const char *dns_name);
00099
00109 bool fileContainsLine(const char *path, const char *expected_line);
00110
00118 int appendLine(const char *path, const char *line);
00119
00120 #endif
```


Index

- Action
 - actions.h, [10](#)
- action
 - Command, [5](#)
- ACTION_ADD
 - actions.h, [10](#)
- ACTION_FLUSH
 - actions.h, [10](#)
- ACTION_INVALID
 - actions.h, [10](#)
- ACTION_PREP
 - actions.h, [10](#)
- ACTION_RESTORE
 - actions.h, [10](#)
- actions.h
 - Action, [10](#)
 - ACTION_ADD, [10](#)
 - ACTION_FLUSH, [10](#)
 - ACTION_INVALID, [10](#)
 - ACTION_PREP, [10](#)
 - ACTION_RESTORE, [10](#)
- addDnsName
 - fix-hosts.c, [11](#)
 - fix-hosts.h, [16](#)
- allow_file
 - Paths, [6](#)
- appendLine
 - system-actions.c, [23](#)
 - system-actions.h, [29](#)
- Command, [5](#)
 - action, [5](#)
 - dns_name, [5](#)
 - verbose, [5](#)
- confirm
 - system-actions.c, [23](#)
 - system-actions.h, [29](#)
- copyFile
 - system-actions.c, [23](#)
 - system-actions.h, [29](#)
- dns_name
 - Command, [5](#)
- ensureDirectory
 - system-actions.c, [24](#)
- system-actions.h, [30](#)
- etc_dir
 - Paths, [6](#)
- fileContainsLine
 - system-actions.c, [24](#)
 - system-actions.h, [30](#)
- fileExists
 - system-actions.c, [25](#)
 - system-actions.h, [30](#)
- fileInfo
 - system-actions.c, [25](#)
 - system-actions.h, [31](#)
- fileType
 - system-actions.c, [25](#)
- fix-hosts.c
 - addDnsName, [11](#)
 - flushDnsCache, [11](#)
 - joinPath, [12](#)
 - lineContainsHost, [12](#)
 - loadPaths, [13](#)
 - removeHostEntry, [13](#)
 - showFileInfo, [14](#)
 - updateHostsFiles, [14](#)
 - usage, [15](#)
- fix-hosts.h
 - addDnsName, [16](#)
 - flushDnsCache, [16](#)
 - loadPaths, [17](#)
 - PATH_MAX, [16](#)
 - removeHostEntry, [17](#)
 - updateHostsFiles, [18](#)
 - usage, [18](#)
- flushDnsCache
 - fix-hosts.c, [11](#)
 - fix-hosts.h, [16](#)
- formatPermissions
 - system-actions.c, [26](#)
- hblock_dir
 - Paths, [6](#)
- hosts_file
 - Paths, [7](#)
- isValidDnsName
 - system-actions.c, [26](#)

- system-actions.h, 31
- joinPath
 - fix-hosts.c, 12
- lineContainsHost
 - fix-hosts.c, 12
- listFiles
 - system-actions.c, 26
 - system-actions.h, 32
- loadPaths
 - fix-hosts.c, 13
 - fix-hosts.h, 17
- main
 - main.c, 20
- main.c
 - main, 20
 - parseArguments, 20
 - programName, 21
 - selectAction, 21
- original_hosts_file
 - Paths, 7
- parseArguments
 - main.c, 20
- PATH_MAX
 - fix-hosts.h, 16
- Paths, 6
 - allow_file, 6
 - etc_dir, 6
 - hblock_dir, 6
 - hosts_file, 7
 - original_hosts_file, 7
- programName
 - main.c, 21
- removeHostEntry
 - fix-hosts.c, 13
 - fix-hosts.h, 17
- REPORT_ERROR
 - system-actions.h, 28
- reportError
 - system-actions.c, 27
 - system-actions.h, 32
- runCommand
 - system-actions.c, 27
 - system-actions.h, 32
- selectAction
 - main.c, 21
- showFileInfo
 - fix-hosts.c, 14
- src/actions.h, 9, 10
- src/fix-hosts.c, 10
- src/fix-hosts.h, 15, 19
- src/main.c, 19
- src/system-actions.c, 22
- src/system-actions.h, 28, 33
- system-actions.c
 - appendLine, 23
 - confirm, 23
 - copyFile, 23
 - ensureDirectory, 24
 - fileContainsLine, 24
 - fileExists, 25
 - fileInfo, 25
 - fileType, 25
 - formatPermissions, 26
 - isValidDnsName, 26
 - listFiles, 26
 - reportError, 27
 - runCommand, 27
- system-actions.h
 - appendLine, 29
 - confirm, 29
 - copyFile, 29
 - ensureDirectory, 30
 - fileContainsLine, 30
 - fileExists, 30
 - fileInfo, 31
 - isValidDnsName, 31
 - listFiles, 32
 - REPORT_ERROR, 28
 - reportError, 32
 - runCommand, 32
- updateHostsFiles
 - fix-hosts.c, 14
 - fix-hosts.h, 18
- usage
 - fix-hosts.c, 15
 - fix-hosts.h, 18
- verbose
 - Command, 5